

SpillGuard Flow - Technical Stack Overview

This document explains the website architecture, technologies used, database design, and how the system works together for maritime oil spill monitoring and investigation.

1. Website / Frontend Stack

- Frontend is built with plain HTML, CSS, and JavaScript instead of a heavy framework like React or Angular.
- The website structure is in `index.html`, `css/style.css`, `js/app.js`, `js/config.js`, and `js/data.js`.
- Leaflet is used for the interactive map, showing maritime regions, spill overlays, vessel routes, and investigation layers.
- Map tiles use MapTiler hybrid imagery and fall back to other map providers if the key is not available.
- The UI is designed as a maritime dashboard: region selector, map panel, spill overview cards, vessel list, and investigation drawers.
- JavaScript fetches data from the backend API and updates the map and panels dynamically.

2. Backend Stack

- The backend is built with Python and FastAPI, which provides a modern API layer and static website hosting.
- FastAPI handles routes such as `/api/bootstrap`, `/api/spills`, `/api/vessels`, `/api/investigations`, `/api/analytics`, and `/api/drift`.
- Uvicorn runs the FastAPI app in development and production-like environments.
- Pydantic models validate request and response payloads for vessel matching, drift calculations, and analyst notes.
- CORS is enabled so the frontend can communicate with the API from the browser.

3. Database Stack

- The project uses PostgreSQL as the main database instead of SQLite.
- The Python driver is psycopg, and the connection string is read from the `DATABASE_URL` environment variable.
- Default connection target is `postgresql://postgres:postgres@localhost:5432/spillguard`
- The database is initialized automatically when the app starts by calling `init_db()` from the API startup event.
- The app creates required tables if they do not already exist and seeds sample data for spills and vessels.

4. Database Tables and Their Use

- `spills`: stores spill incidents such as `id`, `title`, `severity`, `status`, `coast/lat-lon`, `area`, `volume`, `confidence`, `sensor`, `wind/current conditions`, and `polygon geometry`.
- `vessels`: stores vessel metadata and live risk information, including `IMO`, `name`, `MMSI`, `type`, `position`, `heading`, `speed`, `risk score`, and `track history`.

- investigations: stores investigation details, classification, incident metadata, satellite readings, metocean conditions, drift simulation, and candidate suspect data.
- vessel_notes: stores analyst comments attached to specific vessels; this supports operator observations and investigation notes.
- analytics: stores KPI data, monthly trends, regional distributions, and top fleet statistics for analytical dashboards.

5. Why PostgreSQL Is Used

- PostgreSQL gives better support for structured relational data, constraints, and reporting than SQLite for a maritime investigation application.
- The app needs persistent storage for incidents, vessels, investigations, and analyst notes, which is exactly what a production-grade relational database is good at.
- This database structure is scalable and easier to extend when more regions, vessel records, sensor feeds, or analyst workflows are added.
- It is also more appropriate for real-world operational dashboards that may later connect to GIS and analytics workflows.

6. API and Data Flow

- The frontend loads the main dashboard data through the /api/bootstrap endpoint.
- The backend queries PostgreSQL and returns spill, vessel, investigation, and analytics payloads as JSON.
- The frontend updates the map, cards, and tables with this live data without reloading the page.
- The drift analysis route calculates probable movement using vessel heading, speed, wind, and current information.
- Matching and ranking endpoints compare SAR vessel detections to AIS records to identify likely suspect vessels.

7. ML / Detection Layer

- The project includes an ML detection workflow for satellite oil spill detection.
- The detection flow uses a UNet-inspired segmentation model plus DBSCAN clustering for spill object detection.
- Demo mode can work without a trained deep learning model by generating synthetic SAR-style images and running the clustering pipeline.
- The supporting Python libraries include numpy, scikit-learn, and Pillow; PyTorch is used for real model training and inference when available.
- The generated detections are converted into GeoJSON polygons and displayed on the Leaflet map.

8. Core Files in the Project

- server/api.py: main FastAPI server and routes for the application.
- server/database.py: PostgreSQL connection, table creation, seeding, and query functions.
- server/models.py: request validation and data models for API contracts.

- server/ml_detector.py: ML detection and synthetic SAR analysis logic.
- js/app.js: frontend logic for map rendering, data fetching, and dashboard interactions.
- js/data.js: region and spill data definitions used by the interface.
- js/config.js: backend API base URL and MapTiler key configuration.
- index.html: web UI layout and dashboard structure.
- css/style.css: styling and dashboard presentation.

9. Overall Architecture Summary

- Browser loads the page from the frontend files.
- Frontend requests JSON data from the FastAPI backend.
- FastAPI queries PostgreSQL for spill, vessel, investigation, and analytics data.
- The backend may also run ML detection or drift calculations before returning results.
- Map and dashboard interface render the processed output to the operator in real time.

10. In One Line

This website is a Python FastAPI maritime monitoring dashboard with a PostgreSQL database, Leaflet-based geospatial map, and ML-driven spill detection capabilities for oil spill investigation and vessel analysis.

Prepared for SpillGuard Flow / Maritime Oil Spill Monitoring Dashboard